Advanced Machine Learning Framework for Membrane Flux Prediction: A Comprehensive Project Roadmap

1. Introduction

1.1 The Global Imperative for Advanced Separation Processes

The accelerating crisis of global water scarcity, driven by population growth, industrial expansion, and climate change, has necessitated a paradigm shift in water treatment technologies. Membrane-based separation processes, specifically Forward Osmosis (FO) and Membrane Distillation (MD), have emerged as critical components in the nexus of water, energy, and food security. Unlike traditional hydraulic pressure-driven processes like Reverse Osmosis (RO), which dominate the desalination market but suffer from high energy intensity and susceptibility to irreversible fouling, emerging osmotically and thermally driven processes offer the potential for treating high-salinity brines and utilizing low-grade waste heat. However, the industrial-scale deployment of these promising technologies is currently hindered by a lack of robust predictive capabilities regarding their performance under dynamic operating conditions.[1]

The central challenge lies in the accurate prediction of permeate flux—the volumetric rate of water transport across the semi-permeable membrane. Flux is the primary determinant of capital and operational expenditure; however, it is governed by a complex interplay of hydrodynamic conditions, membrane material properties, and solution chemistry. In FO systems, for instance, the effective osmotic driving force is severely compromised by Concentration Polarization (CP)—both Internal (ICP) within the porous support layer and External (ECP) at the membrane surface. These phenomena create a non-linear deviation from ideal theoretical flux, rendering simple linear approximations insufficient.[2]

1.2 Limitations of Mechanistic Modeling

Historically, the membrane science community has relied on mechanistic models derived from

first principles, such as the solution-diffusion model coupled with film theory. While theoretically elegant, these models are predicated on idealized assumptions that rarely hold in complex, real-world scenarios. They typically require the precise determination of intrinsic membrane parameters—water permeability (A), solute permeability (B), and the structural parameter (S). The structural parameter, which characterizes the tortuosity, thickness, and porosity of the support layer, is notoriously difficult to measure directly and is often treated as a fitting parameter derived from flux experiments.[2]

Furthermore, mechanistic models often struggle to account for the transient nature of membrane operations, such as the onset of fouling, compaction of the membrane polymer under pressure, or the complex thermodynamic interactions in multi-component feed solutions. As noted in recent literature, the solution-diffusion model frequently fails to capture the synergistic effects of operating parameters, leading to significant discrepancies between predicted and experimental flux values.[4] This limitation forces plant operators to rely on conservative, sub-optimal operating setpoints, resulting in energy inefficiencies and oversized infrastructure.

1.3 The Data-Driven Paradigm Shift

The convergence of high-throughput experimentation and advanced computational intelligence has catalyzed a transition towards data-driven modeling in membrane science. Machine Learning (ML) and Artificial Intelligence (AI) offer a transformative approach by learning the latent, high-dimensional relationships between input variables (e.g., feed concentration, cross-flow velocity, temperature) and process outputs (flux, rejection) directly from empirical data.[5] Unlike mechanistic models, ML algorithms do not require a priori knowledge of the physical laws governing transport; instead, they act as universal approximators capable of capturing non-linearities and interactions that physical models might overlook.

Recent surveys from 2024 and 2025 indicate that ML models, particularly ensemble tree-based algorithms like XGBoost and CatBoost, are achieving state-of-the-art accuracy in predicting membrane performance, often surpassing Artificial Neural Networks (ANNs) in tabular data tasks.[7] Moreover, the integration of Explainable AI (XAI) techniques, such as SHapley Additive exPlanations (SHAP), has begun to dismantle the "black-box" criticism of ML, offering deep insights into feature importance that align with—and sometimes challenge—established engineering intuition.[4]

1.4 Project Objectives and Scope

This report outlines a comprehensive roadmap for establishing a greenfield ML project dedicated to predicting membrane flux, analyzing parameter influence, and optimizing performance. Leveraging a raw dataset containing experimental records from various studies

(Gray, McCutcheon & Elimelech, etc.), this project aims to build a predictive engine that is not only accurate but also interpretable and actionable.
The specific objectives are:

1. To construct a robust data pipeline that ingests heterogeneous experimental data, handles missing values relating to membrane structural parameters, and engineers physically relevant features.
2. To develop and benchmark advanced ML regression models, specifically focusing on Gradient Boosting Decision Trees (GBDTs) due to their proven efficacy in recent literature.[7]
3. To implement Bayesian Optimization for hyperparameter tuning, moving beyond inefficient grid search methods to find global optima in the model configuration space.[10]
4. To deploy SHAP analysis for dissecting the model's decision-making process, identifying critical control variables, and quantifying their marginal contribution to flux decline or enhancement.[9]
5. To demonstrate an inverse design framework where the trained model is used to prescribe optimal operating conditions for maximizing flux, thereby closing the loop between prediction and process control.

2. Theoretical Framework

2.1 Physics of Osmotically Driven Membrane Processes

To effectively model membrane flux, one must first understand the physical phenomena the ML model attempts to approximate. In Forward Osmosis (FO), water permeates from a feed solution (FS) of low osmotic pressure to a draw solution (DS) of high osmotic pressure across a semi-permeable membrane. The theoretical water flux, J_w, is governed by the general osmotic transport equation:

$$J_w = A (\sigma \Delta \pi - \Delta P)$$
Here, A is the water permeability coefficient of the active layer, σ is the reflection coefficient (typically assumed to be 1 for high-rejection membranes), $\Delta \pi$ is the osmotic pressure difference across the active layer, and ΔP is the applied hydraulic pressure (usually zero in FO).
The critical deviation from this ideal behavior arises from Concentration Polarization (CP). Because the membrane is selective, solute accumulates at the membrane surface on the feed side (concentrative ECP) and is diluted within the porous support layer on the draw side (dilutive ICP). The effective osmotic driving force is thus significantly lower than the bulk osmotic pressure difference ($\Delta \pi_{bulk}$). Mechanistic models attempt to correct for this using the structural parameter S:

$$K_{ICP} = \frac{D}{S} = \frac{D \cdot \epsilon}{\tau \cdot t}$$
where D is the diffusion coefficient of the solute, ϵ is porosity, τ is tortuosity, and t is thickness. The complex dependency of S on membrane fabrication methods and its variability under operation makes it a prime candidate for implicit learning via ML, rather than explicit calculation.[2]

2.2 Machine Learning Algorithms in Membrane Science

While early applications of AI in membrane science relied heavily on simple Feed-Forward Neural Networks (ANNs), the landscape has evolved significantly. The provided research materials highlight a clear hierarchy of model performance for tabular experimental data.

2.2.1 Artificial Neural Networks (ANNs)

ANNs are inspired by biological neural networks and consist of layers of interconnected nodes (neurons). They are exceptionally powerful at capturing non-linear relationships. However, they require large datasets to converge without overfitting and are sensitive to feature scaling. In the context of the provided dataset, which appears to be in the order of hundreds to thousands of data points rather than millions, ANNs may struggle with generalization compared to ensemble methods.[7]

2.2.2 Gradient Boosting Decision Trees (XGBoost, CatBoost, LightGBM)

Recent literature from 2024-2025 consistently positions Gradient Boosting machines as the superior choice for process parameter prediction in FO and MD.[4] These algorithms build an ensemble of weak learners (decision trees) sequentially, where each new tree attempts to correct the errors of the previous ones.

- **XGBoost (Extreme Gradient Boosting):** Renowned for its execution speed and model performance. It includes regularization parameters (L1 and L2) that prevent overfitting—a crucial feature for experimental datasets with noise. It handles missing values internally, which is vital given the gaps in the provided "Raw_data.csv" regarding structural parameters (S).[4]
- **CatBoost:** This algorithm excels at handling categorical variables (like "Membrane Type" or "Flow Orientation") without extensive preprocessing (like One-Hot Encoding). Recent studies by Ibrar et al. (2024) demonstrated CatBoost achieving an R^2 of 0.94 in predicting reverse solute flux, outperforming ANNs.[7]

2.2.3 Deep Learning and Computer Vision

For applications involving membrane morphology (e.g., SEM images of pore structures), Deep Learning models like Convolutional Neural Networks (CNNs) and U-Net architectures are dominant. The reference document provided ("Report-final(New).pdf") focuses on Semantic Segmentation for self-driving cars using U-Net. While this architecture is powerful for image data (segmenting lanes vs. roads), it is generally ill-suited for the tabular regression task of predicting flux from numerical operating parameters. Therefore, while we acknowledge the utility of U-Net in *characterizing* membrane surfaces (e.g., calculating porosity from micrographs), this report will focus on the regression algorithms (XGBoost/BayesOpt) pertinent to the user's goal of *flux prediction* based on the CSV data.[1]

2.3 Bayesian Optimization

Optimization in membrane research has traditionally employed One-Factor-At-a-Time (OFAT) methods or Response Surface Methodology (RSM).[13] However, these methods are inefficient in high-dimensional spaces. Bayesian Optimization (BO) represents the state-of-the-art for hyperparameter tuning and experimental design. BO builds a probabilistic model (surrogate function) of the objective function and uses an acquisition function to decide where to sample next. This balances **exploration** (sampling regions of high uncertainty) and **exploitation** (sampling regions predicted to be optimal). Research indicates BO can be 30 times more efficient than genetic algorithms in converging to optimal solutions for complex design spaces.[11]

3. Literature Review and Data Analysis

3.1 Analysis of the Provided Dataset

The dataset [12] aggregates experimental results from seminal papers in the field, including works by Gray (2006), McCutcheon & Elimelech (2006), and Tang & Ng (2008). A granular analysis of this data reveals several critical clusters and trends that the ML model must capture:

- **Solute Diversity:** The dataset includes a wide range of draw solutions, from inorganic salts (NaCl, MgCl$_2$, NH$_4$HCO$_3$) to organic compounds (Dextrose, Sucrose). This diversity implies that the model must learn to differentiate between solutes not just by concentration, but by their implicit physicochemical properties (diffusivity, van 't Hoff factor) which govern ICP. For instance, sucrose, with a larger hydration radius than

NaCl, induces more severe ICP, leading to lower flux despite high osmotic pressure.[12]

- **Membrane Types:** The data distinguishes between Cellulose Triacetate (CTA) membranes from HTI and Thin Film Composite (TFC) membranes from Oasys. TFC membranes generally exhibit higher water permeability (A) and lower structural parameters (S) than CTA, leading to higher flux. The categorical feature "Membrane AL Type" serves as a proxy for these intrinsic material properties.
- **Operating Conditions:** Variables such as "Flow Orientation" (AL-FS vs. AL-DS) significantly impact flux. The AL-DS (Active Layer facing Draw Solution) mode typically yields higher flux due to reduced ICP compared to AL-FS (PRO mode), a phenomenon the model must learn to predict.[15]

3.2 Emerging Trends in Predictive Modeling (2024-2025)

The literature snippets indicate a rapid maturation of ML applications in this domain.

- **Feature Importance Consensus:** Multiple studies using SHAP analysis have converged on the finding that "Osmotic Pressure Difference" and "Feed Temperature" are the dominant features driving flux, followed by cross-flow velocity.[4] This confirms that data-driven models are correctly identifying the physical driving forces.
- **From Prediction to Design:** The frontier has moved beyond simple prediction. Studies in 2025 are using "Inverse Design," where ML models coupled with genetic algorithms or Bayesian optimization actively suggest new membrane formulations (e.g., monomer ratios) or operating conditions to break the trade-off between permeability and selectivity.[10]
- **Interpretability Wars:** There is an active academic debate regarding the reliability of feature importance rankings. Maliwan et al. (2025) highlight that while parameters like Transmembrane Pressure (TMP) are statistically important, establishing causal links requires careful removal of confounding variables.[9] This suggests our project must be rigorous in its use of SHAP, avoiding causal claims where only correlation exists.

4. Methodology: Building the Advanced Prediction Framework

This section details the step-by-step construction of the machine learning pipeline. The methodology is designed to be modular, robust, and scientifically rigorous, adhering to the best practices identified in the literature review.

4.1 Data Ingestion and Preprocessing

Data quality is the singular most important determinant of model success. The raw data [12] contains categorical strings and potential missing values that must be addressed before any learning can occur.

Feature Engineering Strategy:

1. **Driving Force Calculation:** The core physics of the process is driven by the net osmotic pressure difference. While the dataset provides individual osmotic pressures for feed and draw, an engineered feature Delta_Pi = DS_Osmotic_Pressure - FS_Osmotic_Pressure explicitly represents the driving force, simplifying the learning task for the model.
2. **Dimensional Analysis:** To improve generalization, we can calculate the Reynolds number (Re) for the flow channels if channel geometry data were available. Given the current dataset limitations, we will rely on Velocity as a proxy for hydrodynamic conditions.
3. **Handling Categorical Data:** The variables "Membrane Type" (CTA/TFC), "Flow Orientation" (AL-FS/AL-DS), and "Solutes" (NaCl/Sucrose) contain vital structural information. We will use One-Hot Encoding (OHE) for these. While Label Encoding is simpler, OHE prevents the model from assuming an ordinal relationship (e.g., assuming TFC > CTA numerically) where none exists.

Code Snippet 1: Data Preprocessing Pipeline

Python

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

def load_and_process_data(file_path):
    # Load Data
    try:
        df = pd.read_csv(file_path)
    except:
        # Fallback for different encodings if necessary
        df = pd.read_csv(file_path, encoding='latin1')

    # column name cleaning
    df.columns = df.columns.str.strip()

    # 1. Handling Missing Targets
    # We cannot train on rows with no Flux value
    df = df.dropna(subset=['Membrane Flux (LMH)'])
```

```
    # 2. Feature Engineering
    # Calculate effective driving force.
    # Note: This assumes complete data for pressures. If missing, imputation strategy needed.
    df = df - df

    # 3. Select Features
    # We separate physical conditions (numerical) from system configurations (categorical)
    numeric_features =

    categorical_features =

    target = 'Membrane Flux (LMH)'

    # 4. Fill Numerical Missing Values (Imputation)
    # Using median is safer than mean for outliers
    for col in numeric_features:
        df[col] = df[col].fillna(df[col].median())

    # 5. Encoding Categoricals
    # We use get_dummies for transparency in this example
    df_encoded = pd.get_dummies(df[categorical_features + numeric_features],
                                columns=categorical_features,
                                drop_first=True)

    X = df_encoded
    y = df[target]

    return train_test_split(X, y, test_size=0.2, random_state=42)

# Usage placeholder
# X_train, X_test, y_train, y_test = load_and_process_data('Supplementary Data.xlsx - Raw_data.csv')
```

4.2 Algorithm Selection: The Case for XGBoost

Based on the comparative analysis in recent studies [4], XGBoost is selected as the primary regressor.

- **Justification:** Unlike standard Random Forests, XGBoost utilizes a gradient descent framework to minimize the loss function, correcting errors made by previous trees. It handles sparse data (common after One-Hot Encoding) efficiently. It is also less prone to overfitting than Deep Neural Networks on datasets of this size (<10,000 rows).

- **Alternative:** CatBoost could be used if categorical variables were extremely high-cardinality, but given the limited number of membrane manufacturers and solutes, XGBoost is sufficient and widely supported.

4.3 Hyperparameter Optimization with Bayesian Strategy

Grid Search is computationally expensive and brute-force. Random Search is hit-or-miss. Bayesian Optimization is the most sophisticated approach for this domain.

- **Mechanism:** It builds a Gaussian Process (GP) surrogate model of the objective function (e.g., RMSE vs. Learning Rate).
- **Advantage:** It focuses computational resources on promising areas of the hyperparameter space. For example, if it finds that max_depth=3 performs poorly, it will not waste time testing max_depth=2 or 4 significantly, but rather jump to max_depth=8 to explore.[11]

Code Snippet 2: Bayesian Optimization Implementation

Python

```
import xgboost as xgb
from bayes_opt import BayesianOptimization
from sklearn.metrics import mean_squared_error

def xgb_eval(learning_rate, max_depth, subsample, colsample_bytree, n_estimators):
    # Parameters from Bayesian Opt come as floats, need integer conversion for some
    params = {
        "objective": "reg:squarederror",
        "learning_rate": learning_rate,
        "max_depth": int(max_depth),
        "subsample": subsample,
        "colsample_bytree": colsample_bytree,
        "n_estimators": int(n_estimators),
        "n_jobs": -1,
        "random_state": 42
    }

    # Using Cross-Validation (cv) inside the loop for robustness
    cv_result = xgb.cv(
        params=params,
        dtrain=xgb.DMatrix(X_train, label=y_train),
        num_boost_round=int(n_estimators),
        nfold=5,
```

```
        metrics="rmse",
        early_stopping_rounds=20,
        seed=42,
        verbose_eval=False
    )

    # Bayesian Opt maximizes, so we return negative RMSE
    return -1.0 * cv_result['test-rmse-mean'].iloc[-1]

# Search Space
pbounds = {
    'learning_rate': (0.01, 0.3),
    'max_depth': (3, 12),
    'subsample': (0.6, 1.0),
    'colsample_bytree': (0.6, 1.0),
    'n_estimators': (100, 1000)
}

optimizer = BayesianOptimization(
    f=xgb_eval,
    pbounds=pbounds,
    random_state=42,
    verbose=2
)

# Run Optimization
# optimizer.maximize(init_points=5, n_iter=25)
```

5. Results and Interpretability (XAI)

5.1 Model Performance Analysis

Upon training the XGBoost model with optimal hyperparameters, typical results in this domain yield an R^2 value between 0.92 and 0.98.[4] The Mean Squared Error (MSE) serves as the primary loss metric. A comparison against a baseline Linear Regression model usually reveals the extent of non-linearity in the system.

- **Linear Regression:** Likely $R^2 \approx 0.60-0.75$. Fails to capture the flux saturation at high concentrations caused by severe ICP.

- **XGBoost:** Likely $R^2 > 0.95$. Successfully models the asymptotic flux behavior and the complex interaction between temperature and viscosity.[4]

5.2 SHAP Analysis: Unveiling the Physics

The application of SHAP (SHapley Additive exPlanations) transforms the model from a predictive black box into an analytical tool. SHAP values distribute the prediction output (flux) among the input features based on their contribution, satisfying the properties of local accuracy and consistency.[19]

5.2.1 Global Feature Importance

The summary plot (Beeswarm plot) typically reveals the hierarchy of influence:

1. **Osmotic Pressure Difference ($\Delta \pi$):** As expected physically, this is the primary driver. High values of $\Delta \pi$ correspond to high positive SHAP values (increasing flux).
2. **Draw Solution Concentration:** Closely linked to osmotic pressure, but distinct in its contribution to ICP.
3. **Membrane Orientation:** The transition from AL-FS to AL-DS usually shows a significant jump in SHAP value, quantifying the "ICP mitigation effect" purely from data.[15]
4. **Temperature:** Higher temperatures increase flux (positive SHAP), reflecting the Arrhenius-type dependence of diffusion and viscosity.

5.2.2 Interaction Effects

SHAP dependence plots allow us to visualize interactions. For example, a plot of *Flux vs. DS Concentration* colored by *Temperature* would likely show that the benefit of increasing concentration is amplified at higher temperatures. This confirms that temperature reduces viscosity, thereby mitigating ICP and allowing the higher driving force to be utilized more effectively—a nuanced physical insight derived entirely from data.[21]

Code Snippet 3: SHAP Visualization

Python

```
import shap
import matplotlib.pyplot as plt

# Train final model with best params
```

```
best_params = optimizer.max['params']
best_params['max_depth'] = int(best_params['max_depth'])
best_params['n_estimators'] = int(best_params['n_estimators'])

model = xgb.XGBRegressor(**best_params)
model.fit(X_train, y_train)

# Create Explainer
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# 1. Summary Plot
plt.figure()
shap.summary_plot(shap_values, X_test, show=False)
plt.title("Global Feature Importance (SHAP)")
plt.show()

# 2. Interaction Plot
# Visualizing how DS Molarity affects flux, colored by Temperature
plt.figure()
shap.dependence_plot("DS Molarity (M)", shap_values, X_test,
                     interaction_index="DS Temperature (°C)", show=False)
plt.title("Interaction: Molarity vs Temperature")
plt.show()
```

6. Optimization Loop: Inverse Design

6.1 The Concept of Inverse Design

Traditional engineering optimizes by trial-and-error. Inverse design asks: "Given a desired output (e.g., Flux = 30 LMH), what are the inputs required?" or "What is the maximum possible flux given these constraints?"
We utilize the trained XGBoost model as a surrogate function. Since the XGBoost prediction is virtually instantaneous compared to a laboratory experiment, we can run an optimization algorithm (like Differential Evolution or Powell's method) on the surrogate model to find the global maximum.

6.2 Bayesian Optimization for Process Control

In an operational setting, we can use Bayesian Optimization again, not for hyperparameters, but for **operating conditions**.

- **Objective Function:** The trained XGBoost model predicting flux.
- **Variables:** Feed Velocity (bounded by pump limits), Temperature (bounded by membrane thermal stability), Concentration.
- **Goal:** Maximize Flux while minimizing Energy (approximated by flow rate).

This creates a "Digital Twin" of the membrane setup. An operator can query the system: "What is the optimal flow rate for a feed concentration of 0.5 M?" and the system returns the precise velocity that balances ECP mitigation with pumping costs.[22]

7. Discussion and Future Enhancements

7.1 Discussion

The implementation of this roadmap moves membrane performance analysis from qualitative observations to quantitative precision. The use of XGBoost addresses the non-linear complexities of ICP/ECP that plague mechanistic models. The integration of SHAP values ensures that the model's logic aligns with physical principles, building trust with domain experts.

The comparison of this approach with the "U-Net for Self-Driving Cars" report [12] highlights the distinct nature of domain-specific AI. While computer vision relies on spatial hierarchies in pixel data (U-Net), process engineering relies on tabular interactions of thermodynamic variables (XGBoost). Both share the need for rigorous data preprocessing and hyperparameter tuning, but the architectures diverge significantly to suit the data structure.

7.2 Future Enhancements

1. **Physics-Informed Machine Learning (PIML):** Future iterations could constrain the loss function using the solution-diffusion equation. This ensures that the model never predicts physically impossible states (e.g., negative flux or flux increasing with lower driving force).[23]
2. **Active Learning:** Instead of static dataset training, an active learning loop could be implemented where the model identifies regions of high uncertainty (e.g., behavior of MgCl$_2$ at high temperatures) and suggests specific experiments to be performed in the lab, efficiently expanding the knowledge base.[24]

3. **Transfer Learning:** As noted in snippet [25], transfer learning can be applied to new membrane materials. A model trained on the large "Raw_data.csv" could be fine-tuned on a small dataset of a novel graphene-oxide membrane, significantly accelerating material discovery.

8. Conclusion

This report has laid out a detailed, executable roadmap for applying advanced machine learning to membrane flux prediction. By rigorously integrating data engineering, ensemble learning (XGBoost), Bayesian optimization, and interpretability analysis (SHAP), researchers can transcend the limitations of traditional semi-empirical modeling. The proposed framework provides not just a prediction of flux, but a comprehensive tool for process understanding and optimization, paving the way for more efficient and intelligent water treatment systems. The code artifacts and methodological guidelines provided herein serve as a foundation for immediate implementation and future innovation in the field of membrane science.

9. Data Tables

Table 1: Comparison of Predictive Models for Membrane Flux (Synthesized from Literature)

Model Type	Best For	Pros	Cons	Typical R2 (Flux)
Solution-Diffusion	Theoretical Understanding	Physically derived; interpretable parameters (A, B, S)	Inaccurate with fouling/CP; params difficult to measure	0.70 - 0.85
ANN (Neural Net)	Complex Non-linearities	Universal approximator; handles complex topology	Black box; prone to overfitting on small data	0.88 - 0.95
XGBoost / CatBoost	Tabular Process Data	Handles missing data; built-in regularization; state-of-the-art accuracy	Less effective for image/spatial data	**0.94 - 0.99**
SVM / SVR	Small Datasets	Robust to outliers; effective in	Slow training on large data; hard to	0.85 - 0.92

		high-dimensional space	tune	

Table 2: Key Variables in Provided Dataset and Their Physical Significance

Variable Category	Feature Name	Physical Impact on Flux	Modeling Handling
Driving Force	FS/DS Molarity, Osmotic Pressure	Primary thermodynamic driver ($\Delta \pi$). High correlation with flux.	Numerical Feature
Hydrodynamics	FS/DS Velocity	Mitigates ECP; increases mass transfer coefficient (k).	Numerical Feature
Thermodynamics	FS/DS Temperature	Increases diffusivity (D); decreases viscosity (μ); affects π.	Numerical Feature
Membrane Structure	Membrane AL Type, Orientation	Determines structural parameter (S) and ICP severity.	Categorical (One-Hot)
Chemistry	Solute Type (NaCl, Dextrose)	Determines diffusion coefficient (D) and hydration radius.	Categorical / Engineered

Table 3: Summary of Optimization Algorithms

Algorithm	Method	Efficiency	Best Use Case
Grid Search	Exhaustive	Low	Low-dimensional space; verifying localized behavior.
Random Search	Stochastic	Medium	Baseline tuning; high-dimensional space.

Genetic Algorithm (GA)	Evolutionary	Medium-High	Complex landscapes; prone to local minima in flat basins.
Bayesian Optimization	Probabilistic	**High**	Expensive objective functions; finding global optima efficiently.

Works cited

1. Utilising Artificial Intelligence to Predict Membrane Behaviour in Water Purification and Desalination - MDPI, accessed on November 22, 2025, https://www.mdpi.com/2073-4441/16/20/2940
2. Forward Osmosis: A Critical Review - MDPI, accessed on November 22, 2025, https://www.mdpi.com/2227-9717/8/4/404
3. Modeling and Optimization of the Forward Osmosis Process - Parameters Selection, Flux Prediction and Process Applications, accessed on November 22, 2025, https://scholarbank.nus.edu.sg/entities/publication/41a0695c-f15b-4f6c-89be-cc9af79c8ad9
4. Modeling and Evaluation of the Permeate Flux in Forward Osmosis Process with Machine Learning | Industrial & Engineering Chemistry Research - ACS Publications, accessed on November 22, 2025, https://pubs.acs.org/doi/abs/10.1021/acs.iecr.2c03064
5. Machine learning algorithms for predicting membrane bioreactors performance: A review, accessed on November 22, 2025, https://pubmed.ncbi.nlm.nih.gov/40101485/
6. Application of machine learning models for predicting membrane permeability under different manufacturing and operational conditions | Request PDF - ResearchGate, accessed on November 22, 2025, https://www.researchgate.net/publication/394821673_Application_of_machine_learning_models_for_predicting_membrane_permeability_under_different_manufacturing_and_operational_conditions
7. A machine learning approach for prediction of reverse solute flux in forward osmosis | Request PDF - ResearchGate, accessed on November 22, 2025, https://www.researchgate.net/publication/371761975_A_machine_learning_approach_for_prediction_of_reverse_solute_flux_in_forward_osmosis
8. xgboost and neural networks : r/learnmachinelearning - Reddit, accessed on November 22, 2025, https://www.reddit.com/r/learnmachinelearning/comments/101ay0o/xgboost_and_neural_networks/
9. Limitations of SHAP-based interpretations in environmental and membrane filtration applications, accessed on November 22, 2025, https://neuro.musashino-u.ac.jp/publications/pdf/watershap.pdf
10. Revolutionizing Membrane Design Using Machine Learning-Bayesian

Optimization, accessed on November 22, 2025, https://pubmed.ncbi.nlm.nih.gov/34968041/
11. bayesian-optimization/BayesianOptimization: A Python implementation of global optimization with gaussian processes. - GitHub, accessed on November 22, 2025, https://github.com/bayesian-optimization/BayesianOptimization
12. Supplementary Data.xlsx
13. Modeling and Sensitivity Analysis of the Forward Osmosis Process to Predict Membrane Flux Using a Novel Combination of Neural Network and Response Surface Methodology Techniques - MDPI, accessed on November 22, 2025, https://www.mdpi.com/2077-0375/11/1/70
14. Bayesian Optimization for Design of Multiscale Biological Circuits - PMC - PubMed Central, accessed on November 22, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC10367132/
15. Optimization of Operating Conditions in Forward Osmosis for Osmotic Membrane Bioreactor, accessed on November 22, 2025, https://openchemicalengineeringjournal.com/VOLUME/3/PAGE/27/ABSTRACT/
16. Permeation Flux Prediction of Vacuum Membrane Distillation Using Hybrid Machine Learning Techniques - PMC - NIH, accessed on November 22, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC10744684/
17. Leveraging machine learning with Bayesian optimization for optimizing feed spacer geometry using insights from predicting Sherwood and Power numbers - ResearchGate, accessed on November 22, 2025, https://www.researchgate.net/publication/396602100_Leveraging_machine_learning_with_Bayesian_optimization_for_optimizing_feed_spacer_geometry_using_insights_from_predicting_Sherwood_and_Power_numbers
18. Limitations of SHAP-based interpretations in environmental and membrane filtration applications - PubMed, accessed on November 22, 2025, https://pubmed.ncbi.nlm.nih.gov/41092570/
19. How to interpret machine learning (ML) models with SHAP values - Mage AI, accessed on November 22, 2025, https://www.mage.ai/blog/how-to-interpret-explain-machine-learning-models-using-shap-values
20. A Unified Approach to Interpreting Model Predictions, accessed on November 22, 2025, https://proceedings.neurips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.pdf
21. Full article: Developing an efficient explainable artificial intelligence approach for accurate reverse osmosis desalination plant performance prediction: application of SHAP analysis, accessed on November 22, 2025, https://www.tandfonline.com/doi/full/10.1080/19942060.2024.2422060
22. Forward osmosis system analysis for optimum design and operating conditions - PubMed, accessed on November 22, 2025, https://pubmed.ncbi.nlm.nih.gov/30189398/
23. Machine learning-guided performance prediction of forward osmosis polymeric membranes for boron recovery - ResearchGate, accessed on November 22,

2025, https://www.researchgate.net/publication/391036696_Machine_learning-guided_performance_prediction_of_forward_osmosis_polymeric_membranes_for_boron_recovery
24. Bayesian Optimization in Bioprocess Engineering—Where Do We Stand Today? - PMC, accessed on November 22, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC12067035/
25. Martin Fitzner "Industrial view on Bayesian optimization A perfect match for the low/no-data regime" - YouTube, accessed on November 22, 2025, https://www.youtube.com/watch?v=OKHcwtefRsU